Reflection - Static Code Analysis

1. Which tool helped the most and why?

Pylint helped the most because it identified several code quality and maintainability issues, including missing docstrings, mutable default arguments, naming conventions, and problematic exception handling. Its scoring system also helped measure the improvement in code quality, from 6.32/10 to 10.00/10.

2. Were there any false positives?

Some warnings required interpretation in the context of the program. For example, exception handling should not always be removed; instead, specific exceptions should be handled where appropriate. Static analysis findings must therefore be reviewed before making changes.

3. What did you learn about writing secure code?

I learned that secure code requires more than correct functionality. Using security-oriented random number generation, validating inputs, handling exceptions appropriately, and managing files with context managers can improve security and reliability. Bandit helped identify security-related concerns that might otherwise be overlooked.

4. How would you integrate static analysis into a CI/CD pipeline?

I would configure a CI/CD pipeline to run Pylint, Bandit, and Flake8 automatically whenever code is pushed or a pull request is created. The pipeline could fail if security findings, formatting violations, or code quality scores do not meet the project's requirements. This would help detect problems early and maintain consistent code quality throughout development.